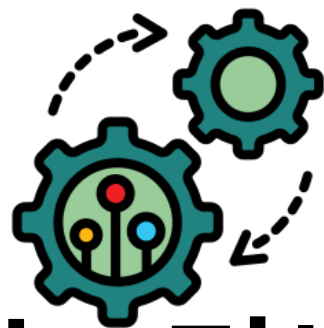




케이녹 리버싱 5회차 과제

17기 최동현 (202511688)

1. PE 파일 포맷 개요

• PE (Portable Executable)

： PE(Portable Executable)는 Windows 운영체제 환경에서 사용되는 실행 파일(EXE, DLL, SYS, OBJ)의 데이터 구조 포맷입니다. PE 파일은 디스크에 저장된 정적 상태의 파일 구조와, OS 로더에 의해 가상 메모리에 적재된 후의 동적 상태 구조를 모두 포함하는 명세서입니다.

** PE 형식의 특징*

- **대상 확장자** : .exe, .dll, .sys(커널 드라이버), .ocx(ActiveX), .scr(스크린세이버), .cpl(제어판 항목)이 모두 동일한 PE 형식을 따름
- **기반 규격** : Unix 계열에서 사용하던 COFF(Common Object File Format)를 기반으로 마이크로소프트가 32비트 Windows(Windows NT)부터 채택
- **로더의 역할** : Windows 로더(Loader)가 파일을 실행할 때 PE 헤더를 해석하여 코드와 데이터를 메모리에 매핑(Mapping)하고, 실행 시작 주소를 결정

2. PE 파일 전체 구조

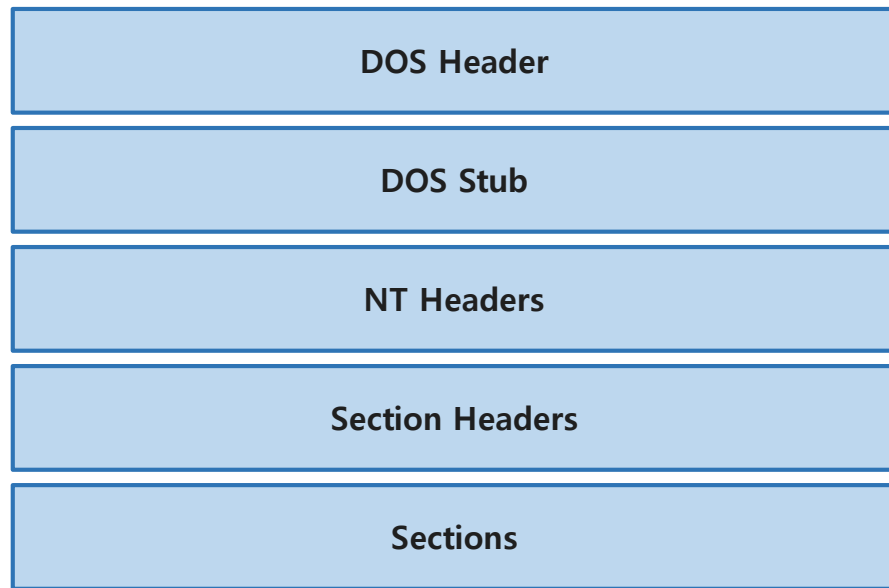
• PE 파일 구성 요소

: PE 파일은 아래 다섯 가지 요소가 순서대로 이어진 구조를 가집니다.

* 구성 순서

- **DOS Header** : IMAGE_DOS_HEADER, MZ 시그니처(0x5A4D)
- **DOS Stub** : "cannot be run in DOS mode" 출력 코드
- **NT Headers** : PE 시그니처 + File Header + Optional Header
- **Section Headers** : 각 섹션의 이름·크기·주소·권한 정의 배열
- **Sections** : .text .data .rdata .rsrc .reloc 실제 데이터

파일 오프셋 기준 순서



* 맵핑 유의사항

- 섹션마다 FileAlignment(디스크, 보통 512B)와 SectionAlignment(메모리, 보통 4096B) 단위로 정렬되어 디스크 크기와 메모리 크기가 달라집니다.

3. DOS Header & NT Headers 상세

- **IMAGE_DOS_HEADER**

- **e_magic** : 파일 시작 2바이트, 'MZ'(0x5A4D) 고정값으로 DOS 실행 파일임을 표시
- **e_lfanew** : 오프셋 0x3C에 위치, NT Headers가 시작되는 파일 오프셋을 저장

- **IMAGE_NT_HEADERS**

- **Signature** : 'PE\0\0'(0x00004550) — NT 헤더의 시작을 확인하는 값
- **FILE_HEADER** : Machine, NumberOfSections, TimeDateStamp, Characteristics 등 파일 전반 정보
- **OPTIONAL_HEADER** : AddressOfEntryPoint, ImageBase, SizeOfImage, DataDirectory[16] 등 로딩·실행 정보

4. Section Header 및 주요 섹션

• IMAGE_SECTION_HEADER

- **Name** : 8바이트 섹션 이름 (예: .text)
- **VirtualSize / VirtualAddress** : 메모리 상 크기·RVA
- **SizeOfRawData / PointerToRawData** : 디스크 상 크기·오프셋
- **Characteristics** : 읽기·쓰기·실행 권한 플래그

섹션	내용	권한
.text	코드(Instruction)	실행 (R-X)
.data	초기화된 전역/정적 변수	읽기·쓰기
.rdata	읽기 전용 데이터, Import/Export 테이블	읽기 전용
.rsrc	아이콘·문자열 등 리소스	읽기 전용
.reloc	재배치(주소 보정) 정보	읽기 전용

5. IAT (Import Address Table)

- IAT (Import Address Table)

: 실행 파일이 다른 DLL의 함수를 호출할 때, 로더가 채워 넣는 실제 함수 주소 배열입니다.

* 구조

- **Import Table** : Data Directory 인덱스 1, IMAGE_IMPORT_DESCRIPTOR 배열 (DLL마다 하나씩)
- **OriginalFirstThunk (INT)** : 함수 이름을 가리키는 배열, 로딩 후에도 값이 변하지 않음
- **FirstThunk (IAT)** : 함수 이름을 가리키던 값이 로딩 시 실제 함수 '주소'로 교체되는 배열
- **간접 호출** : 코드는 'call dword ptr [IAT_entry]'처럼 IAT를 경유해 함수를 간접 호출

6. EAT (Export Address Table)

- EAT (Export Address Table)

: DLL이 다른 실행 파일에게 함수를 제공한다고 공개하는 테이블이며, Data Directory 인덱스 0이 가리킵니다. 라이브러리에서 Export 해주는 함수의 시작 주소를 정확히 구할 수 있습니다.

- **AddressOfFunctions** : 실제 함수 주소 배열 (EAT 본체)
- **AddressOfNames / AddressOfNameOrdinals** : 함수 이름 문자열과 이름→순번 매핑
- **조회 과정** : 이름 검색 → Ordinal 확인 → AddressOfFunctions[순번]에서 주소 획득 (GetProcAddress 내부 동작)

7-1. Dreamhack (rev-basic-6)

IDA

```
IDA View-A Pseudocode-A Hex
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     printf((__int64)"Input : ", argv, envp);
7     scanf("%256s", v4);
8     if ( (unsigned int)sub_7FF65DA61000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

```
IDA View-A Pseudocode-A Hex View-1
1 int64 __fastcall sub_7FF65DA61000(int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned int64)i < 0x12; ++i )
6     {
7         if ( byte_7FF65DA63020[(unsigned __int8 *)(a1 + i)] != byte_7FF65DA63000[i] )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

```
for(int i = 0; i < 18; i++) {
    if ( arr1[a1 + i] != arr2[i] ) {
        return 0;
    }
    return 1;
}
```

V4 값을 입력받음, sub_7FF65DA61000함수 return 값이 1이면
Corret표시를 함.

7-2. Dreamhack (rev-basic-6)

IDA

```
.data:00007FF65DA63000 ; unsigned __int8 byte_7FF65DA63000[32]
. data:00007FF65DA63000 byte_7FF65DA63000 db 0, 4Dh, 51h, 50h, 0EFh, 0FBh, 0C3h, 0CFh, 92h, 45h
. data:00007FF65DA63000 ; DATA XREF: sub_7FF65DA61000+40fo
. data:00007FF65DA6300A db 4Dh, 0CFh, 0F5h, 4, 40h, 50h, 43h, 63h, 0Eh dup(0)
. data:00007FF65DA63020 ; unsigned __int8 byte_7FF65DA63020[256]
. data:00007FF65DA63020 byte_7FF65DA63020 db 63h, 7Ch, 77h, 7Bh, 0F2h, 6Bh, 6Fh, 0C5h, 30h, 1, 67h
. data:00007FF65DA63020 ; DATA XREF: sub_7FF65DA61000+31fo
. data:00007FF65DA6302B db 2Bh, 0FEh, 0D7h, 0ABh, 76h, 0CAh, 82h, 0C9h, 7Dh, 0FAh
. data:00007FF65DA63035 db 59h, 47h, 0F0h, 0ADh, 0D4h, 0A2h, 0AFh, 9Ch, 0A4h, 72h
. data:00007FF65DA6303F db 0C0h, 0B7h, 0FDh, 93h, 26h, 36h, 3Fh, 0F7h, 0CCh, 34h
. data:00007FF65DA63049 db 0A5h, 0E5h, 0F1h, 71h, 0D8h, 31h, 15h, 4, 0C7h, 23h
. data:00007FF65DA63053 db 0C3h, 18h, 96h, 5, 9Ah, 7, 12h, 80h, 0E2h, 0EBh, 27h
. data:00007FF65DA6305E db 0B2h, 75h, 9, 83h, 2Ch, 1Ah, 1Bh, 6Eh, 5Ah, 0A0h, 52h
. data:00007FF65DA63069 db 3Bh, 0D6h, 0B3h, 29h, 0E3h, 2Fh, 84h, 53h, 0D1h, 0
. data:00007FF65DA63073 db 0EDh, 20h, 0FCh, 0B1h, 5Bh, 6Ah, 0CBh, 0BEh, 39h, 4Ah
. data:00007FF65DA6307D db 4Ch, 58h, 0CFh, 0D0h, 0EFh, 0AAh, 0FBh, 43h, 4Dh, 33h
. data:00007FF65DA63087 db 85h, 45h, 0F9h, 2, 7Fh, 50h, 3Ch, 9Fh, 0A8h, 51h, 0A3h
. data:00007FF65DA63092 db 40h, 8Fh, 92h, 9Dh, 38h, 0F5h, 0BCh, 0B6h, 0DAh, 21h
. data:00007FF65DA6309C db 10h, 0FFh, 0F3h, 0D2h, 0CDh, 0Ch, 13h, 0ECh, 5Fh, 97h
. data:00007FF65DA630A6 db 44h, 17h, 0C4h, 0A7h, 7Eh, 3Dh, 64h, 5Dh, 19h, 73h
. data:00007FF65DA630B0 db 60h, 81h, 4Fh, 0DCh, 22h, 2Ah, 90h, 88h, 46h, 0EEh
. data:00007FF65DA630BA db 0B8h, 14h, 0DEh, 5Eh, 0Bh, 0DBh, 0E0h, 32h, 3Ah, 0Ah
. data:00007FF65DA630C4 db 49h, 6, 24h, 5Ch, 0C2h, 0D3h, 0ACh, 62h, 91h, 95h, 0E4h
. data:00007FF65DA630CF db 79h, 0E7h, 0C8h, 37h, 6Dh, 8Dh, 0D5h, 4Eh, 0A9h, 6Ch
. data:00007FF65DA630D9 db 56h, 0F4h, 0EAh, 65h, 7Ah, 0AEh, 8, 0BAh, 78h, 25h
. data:00007FF65DA630E3 db 2Eh, 1Ch, 0A6h, 0B4h, 0C6h, 0E8h, 0DDh, 74h, 1Fh, 4Bh
. data:00007FF65DA630ED db 0BDh, 8Bh, 8Ah, 70h, 3Eh, 0B5h, 66h, 48h, 3, 0F6h, 0Eh
. data:00007FF65DA630F8 db 61h, 35h, 57h, 0B9h, 86h, 0C1h, 1Dh, 9Eh, 0E1h, 0F8h
. data:00007FF65DA63102 db 98h, 11h, 69h, 0D9h, 8Eh, 94h, 9Bh, 1Eh, 87h, 0E9h
. data:00007FF65DA6310C db 0CEh, 55h, 28h, 0DFh, 8Ch, 0A1h, 89h, 0Dh, 0BFh, 0E6h
. data:00007FF65DA63116 db 42h, 68h, 41h, 99h, 2Dh, 0Fh, 0B0h, 54h, 0BBh, 16h
```

위 계산식의 배열1과 2의 배열 값들이 있다.

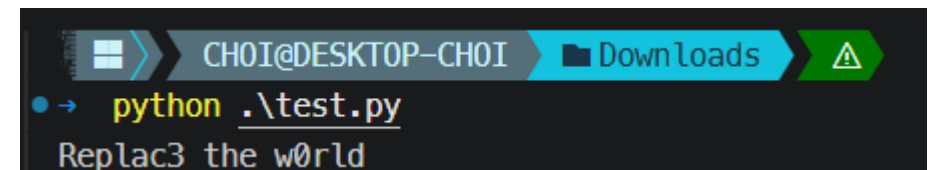
7-3. Dreamhack (rev-basic-6)

Script

```
C: > Users > CHOI > Downloads > test.py
1  arr1 = ['63', '7C', '77', '7B', 'F2', '6B', '6F', 'C5', '30', '1', '67', '2B', 'FE', 'D7', 'AB', '76',
2  'CA', '82', 'C9', '7D', 'FA', '59', '47', 'F0', 'AD', 'D4', 'A2', 'AF', '9C', 'A4', '72', 'C0',
3  'B7', 'FD', '93', '26', '36', '3F', 'F7', 'CC', '34', 'A5', 'E5', 'F1', '71', 'D8', '31', '15',
4  '4', 'C7', '23', 'C3', '18', '96', '5', '9A', '7', '12', '80', 'E2', 'EB', '27', 'B2', '75',
5  '9', '83', '2C', '1A', '1B', '6E', '5A', 'A0', '52', '3B', 'D6', 'B3', '29', 'E3', '2F', '84',
6  '53', 'D1', '0', 'ED', '20', 'FC', 'B1', '5B', '6A', 'CB', 'BE', '39', '4A', '4C', '58', 'CF',
7  'D0', 'EF', 'AA', 'FB', '43', '4D', '33', '85', '45', 'F9', '2', '7F', '50', '3C', '9F', 'A8',
8  '51', 'A3', '40', '8F', '92', '9D', '38', 'F5', 'BC', 'B6', 'DA', '21', '10', 'FF', 'F3', 'D2',
9  'CD', '0C', '13', 'EC', '5F', '97', '44', '17', 'C4', 'A7', '7E', '3D', '64', '5D', '19', '73',
10 '60', '81', '4F', 'DC', '22', '2A', '90', '88', '46', 'EE', 'B8', '14', 'DE', '5E', '0B', 'DB',
11 'E0', '32', '3A', '0A', '49', '6', '24', '5C', 'C2', 'D3', 'AC', '62', '91', '95', 'E4', '79',
12 'E7', 'C8', '37', '6D', '8D', 'D5', '4E', 'A9', '6C', '56', 'F4', 'EA', '65', '7A', 'AE', '8',
13 'BA', '78', '25', '2E', '1C', 'A6', 'B4', 'C6', 'E8', 'DD', '74', '1F', '4B', 'BD', '8B', '8A' ]
14
15  arr2 = ['0', '4D', '51', '50', 'EF', 'FB', 'C3', 'CF', '92', '45', '4D', 'CF', 'F5', '4', '40', '50', '43', '63']
16
17  flag = ''
18
19  for i in range(len(arr2)):
20      for j in range(len(arr1)):
21          if arr2[i] == arr1[j]:
22              flag += chr(j)
23
24  print(flag)
```

```
for(int i = 0; i < 18; i++) {
    if ( arr1[a1 + i] == arr2[i] ) {
        return 1;
    }
}
```

프로그램의 슈도코드를 값을 역산하여 arr2[i] == arr[j]가 일치
한다면 플래그 문자열에 ASCII값을 추가해준다.



8-1. reversing.kr (Easy Keygen)

Find the Name when the Serial is 5B134977135E7D13

Readme파일에 Serial이 5B134977135E7D13일때 이름을 찾으라고 힌트가 나와있다.

```
CH0I@DESKTOP-CH0I Easy_KeygenMe
> & '.\Easy Keygen.exe'
Input Name: test
Input Serial: 5B134977135E7D13
Wrong
```

실행해보니 이름이 틀렸는지 wrong이라고 뜨는것을 볼 수 있다.

0040102E	push	easy	keygen.408060	00408060	"Input Name: "
00401054	push	easy	keygen.40805C	0040805C	"%s"
00401094	push	easy	keygen.408054	00408054	"%s%02X"
004010C1	push	easy	keygen.408044	00408044	"Input Serial: "
004010D5	push	easy	keygen.40805C	0040805C	"%s"
0040111A	push	easy	keygen.408038	00408038	"Correct!\n"
00401130	push	easy	keygen.408030	00408030	"Wrong\n"

디버거로 실행해 문자열을 참조했을때 이름 입력, 시리얼 입력, 정답 및 실패 메시지가 보임
Input Name: 코드로 이동하여 보았을때 아래처럼 보인다.

0040102E	68 60804000	push	easy	keygen.408060	408060:"Input Name: "
00401033	F3:AB	rep	stosd		
00401035	66:AB	stosw			
00401037	AA	stosb			
00401038	C64424 10 10	mov	byte	ptr	ss:[esp+10],10
0040103D	C64424 11 20	mov	byte	ptr	ss:[esp+11],20
00401042	C64424 12 30	mov	byte	ptr	ss:[esp+12],30
00401047	E8 6D010000	call	easy	keygen.4011B9	20:' '
					30:'0'

8-1. reversing.kr (Easy Keygen)

0040102E	68 60804000	push easy keygen.408060	408060:"Input Name: "
00401033	F3:AB	rep stosd	
00401035	66:AB	stosw	
00401037	AA	stosb	
00401038	C64424 10 10	mov byte ptr ss:[esp+10],10	20:' '
0040103D	C64424 11 20	mov byte ptr ss:[esp+11],20	30:'0'
00401042	C64424 12 30	mov byte ptr ss:[esp+12],30	
00401047	E8 6D010000	call easy keygen.4011B9	

함수 처음 위와 같은 명령에서 스택 세그먼트에 고정값 세개 10h,20h,30h이 저장됩니다.
그 다음 계산 구간에선 name의 글자를 키 10h,20h,30h과 순서대로 XOR 하게 됩니다.

00401077	83FE 03	cmp esi,3	esi:EntryPoint
0040107A	7C 02	j1 easy keygen.40107E	
0040107C	33F6	xor esi,esi	esi:EntryPoint
0040107E	0FB E4C34 0C	movsx ecx,byte ptr ss:[esp+esi+C]	ecx:EntryPoint
00401083	0FB E542C 10	movsx edx,byte ptr ss:[esp+ebp+10]	edx:EntryPoint
00401088	33CA	xor ecx,edx	ecx:EntryPoint, edx:EntryPoint
0040108A	8D4424 74	lea eax,dword ptr ss:[esp+74]	
0040108E	51	push ecx	ecx:EntryPoint
0040108F	50	push eax	
00401090	8D4C24 7C	lea ecx,dword ptr ss:[esp+7C]	ecx:EntryPoint, [esp+7C]:EntryPoint
00401094	68 54804000	push easy keygen.408054	408054:"%s%02X"

위에서부터 키번호 3인지 확인하고 3이면 xor로 0되고 [esp+esi+C]와 [esp+ebp+10] 을 서로 XOR하는 것을 알수있습니다. 그것을 401094에서 "%s%02X" 형식으로 저장합니다.
근데 여기서 esp+esi+C는 아래 push 408060을 통해 감소되어 esp기준 esp+C인것을 알수있다.

0040104C	83C4 04	add esp,4	
----------	---------	-----------	--

근데 여기서 esi가 붙는데 위의 계산식으로 3보다 작다면 xor를 건너뛰고 3이거나 3보다 크면 esi를 0으로 두기때문에 esi는 0 -> 1 -> 2 -> 0 -> 1 .. 형식으로 3을 넘을 수 없다. 그렇기에 key[i % 3]으로 볼 수 있다.

8-1. reversing.kr (Easy Keygen)

00401077	83FE 03	cmp esi,3	esi:EntryPoint
0040107A	7C 02	j1 easy keygen.40107E	
0040107C	33F6	xor esi,esi	esi:EntryPoint
0040107E	0FB E4C34 0C	movsx ecx,byte ptr ss:[esp+esi+C]	ecx:EntryPoint
00401083	0FB E542C 10	movsx edx,byte ptr ss:[esp+ebp+10]	edx:EntryPoint
00401088	33CA	xor ecx,edx	ecx:EntryPoint, edx:EntryPoint
0040108A	8D4424 74	lea eax,dword ptr ss:[esp+74]	
0040108E	51	push ecx	ecx:EntryPoint
0040108F	50	push eax	
00401090	8D4C24 7C	lea ecx,dword ptr ss:[esp+7C]	ecx:EntryPoint, [esp+7C]:EntryPoint
00401094	68 54804000	push easy keygen.408054	408054:"%s%02X"

0040104F	8D4424 10	lea eax,dword ptr ss:[esp+10]	[esp+10]:RtlInitializeExceptionChain+6B
00401053	50	push eax	
00401054	68 5C804000	push easy keygen.40805C	40805C:"%s"
00401059	E8 44010000	call easy keygen.4011A2	

다음으로 esp+ebp+10같은경우, 위의 lea eax,esp+10을 통해 eax = esp+10주소를 넣고 push eax, push "%s"를 보면 printf나 scanf같은 함수 호출 하는것으로 예측할 수 있는데, input name부분으로 봐서 scanf로 name을 입력하는것으로 알수있다.

0040106A	33ED	xor ebp,ebp	
004010A2	45	inc ebp	

추가적으로 위 명령어를 통해 0에서 1씩 추가하는것을 알수있어 name[i]로 반복 계산하는것을 유추했다.

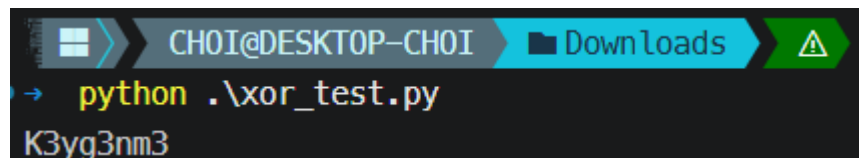
8-1. reversing.kr (Easy Keygen)

```
sub_4011B9(aInputName);
scanf("%s", v8);
v3 = 0;
for ( i = 0; v3 < (int)strlen(v8); ++i )
{
    if ( i >= 3 )
        i = 0;
    sprintf(Buffer, "%s%02X", Buffer, v8[v3++] ^ v7[i - 1]);
}
memset(v8, 0, sizeof(v8));
sub_4011B9(aInputSerial);
scanf("%s", v8);
if ( !strcmp(v8, Buffer) )
    sub_4011B9(aCorrect);
```

이쯤에서 IDA 사용해 소스코드를 확인했는데, 위 처럼 되어있었고 키값과 이름값이 서로 XOR되어 시리얼 값으로 계산되는듯했습니다. XOR값은 한번더 XOR하면 원래 값이 되기에 $\text{name}[i] = \text{serial}[i] \oplus \text{key}[i \% 3]$ 하여 계산하면 역산되어 이름이 나올것입니다.

```
1 serial_text = "58134977135E7D13"
2 serial_value = bytes.fromhex(serial_text)
3
4 key_value = [0x10, 0x20, 0x30]
5 name = ""
6
7 for i in range(len(serial_value)):
8     serial = serial_value[i]
9     key = key_value[i % 3]
10    name_value = serial ^ key
11    name_chr = chr(name_value)
12    name = name + name_chr
13
14 print(name)
```

때문에 옆처럼 name값을 $\text{serial}[i] \oplus \text{key}[i \% 3]$ 를 통해 나온 값을 char로 출력하니 해당 시리얼의 Name이 도출되었습니다.



```
CHOI@DESKTOP-CHOI Downloads
python .\xor_test.py
K3yg3nm3
```

9-1. CodeEngn (basic L16)

```
ReWrit's Crackme#5
*****
* This is my 5th crackme, *
* i hope you will enjoy it. *
*****

Enter your Name: test
Enter your Password: test

Wrong password!
=|
```

16.Exe를 실행하니까 이름하고 비밀번호 입력칸이 나오고 다음 결과가 나오는데 이름하고 패스워드를 찾는 문제인가 생각이들어 IDA랑 x64dbg로 분석을 시작

```
if ( v12 == v13 )
{
    v8 = GetStdHandle(0xFFFFFFFF);
    SetConsoleTextAttribute(v8, 0xAu);
    std::ostream::operator<<(&std::cout, std::endl<char,std::char_traits<char>>);
    std::operator<<<std::char_traits<char>>((int)&std::cout, " Good Job!\n");
    std::operator<<<std::char_traits<char>>((int)&std::cout, " =)");
}
```

보니까 c++인것같았고 v12 == v13일때 성공인것을확인했습니다.

9-1. CodeEngn (basic L16)

004014BF	C74424 04 AF004400	mov dword ptr ss:[esp+4],16.4400AF	4400AF:" Enter your Name: "
004014C7	C70424 C0334400	mov dword ptr ss:[esp],16.4433C0	4433C0:&"췄B"
004014CE	E8 05AF0300	call 16.43C3D8	
004014D3	C745 E4 00000000	mov dword ptr ss:[ebp-1C],0	
004014DA	C745 E0 00000000	mov dword ptr ss:[ebp-20],0	
004014E1	8D45 C8	lea eax,dword ptr ss:[ebp-38]	[ebp-38]:"test"

00401522	8D45 C8	lea eax,dword ptr ss:[ebp-38]	[ebp-38]:"test"
00401525	890424	mov dword ptr ss:[esp],eax	
00401528	E8 030F0100	call 16.412430	
0040152D	3B45 E0	cmp eax,dword ptr ss:[ebp-20]	
00401530	76 0E	jbe 16.401540	
00401532	8D45 C8	lea eax,dword ptr ss:[ebp-38]	[ebp-38]:"test"
00401535	890424	mov dword ptr ss:[esp],eax	
00401538	E8 F30E0100	call 16.412430	
0040153D	8945 E0	mov dword ptr ss:[ebp-20],eax	

Enter your name값을 입력하였을때 ebp-38에 입력한 name값이 들어가는것을 확인하였습니다. 그 다음 두번째에서 412430이라는 함수가 호출되는데 저 함수는 name의 길이를 파악하는 함수라고 알게되었습니다. Return된 eax값이 4일때 ebp-20이 4를 반환하는걸로 봐서 ebp-20가 name의 길이를 저장하는 변수라고 봤습니다.

00401540	8B55 E0	mov edx,dword ptr ss:[ebp-20]	
00401543	89D0	mov eax,edx	
00401545	01C0	add eax,eax	
00401547	01D0	add eax,edx	
00401549	C1E0 02	shl eax,2	
0040154C	8945 C4	mov dword ptr ss:[ebp-3C],eax	

그리고 다음을 통해 add eax,eax add eax,edx, 노 eax,2를 통하여 길이를 통해 계산된 값을 ebp-3C에 저장하는것을 확인했습니다.

9-1. CodeEngn (basic L16)

00401560	C74424 04 C2004400	mov dword ptr ss:[ebp+3C],eax	4400C2:" Enter your Password: "
00401568	C70424 C0334400	mov dword ptr ss:[esp+4],16.4433C0	4433C0:&"쑤B"
0040156F	C745 90 01000000	mov dword ptr ss:[ebp-70],1	
00401576	E8 5DAE0300	call 16.43C3D8	
0040157B	8D45 C0	lea eax,dword ptr ss:[ebp-40]	
0040157E	894424 04	mov dword ptr ss:[esp+4],eax	
00401582	C70424 60344400	mov dword ptr ss:[esp],16.443460	
00401589	E8 B2740200	call 16.428A40	
0040158E	8B45 C4	mov eax,dword ptr ss:[ebp-3C]	
00401591	69D0 80CE0A00	imul edx,eax,ACE80	
00401597	8D45 C4	lea eax,dword ptr ss:[ebp-3C]	
0040159A	0110	add dword ptr ds:[eax],edx	
0040159C	8B45 C0	mov eax,dword ptr ss:[ebp-40]	
0040159F	3B45 C4	cmp eax,dword ptr ss:[ebp-3C]	
004015A2	0F85 94000000	jne 16.40163C	
004015A8	C70424 F5FFFFFF	mov dword ptr ss:[esp],FFFFFFF5	
004015AF	E8 8CF60000	call <JMP.&GetStdHandle>	
004015B4	83EC 04	sub esp,4	
004015B7	C74424 04 0A000000	mov dword ptr ss:[esp+4],A	0A:'\n'
004015BF	890424	mov dword ptr ss:[esp],eax	
004015C2	E8 89F60000	call <JMP.&SetConsoleTextAttribute>	
004015C7	83EC 08	sub esp,8	
004015CA	C74424 04 A8B14300	mov dword ptr ss:[esp+4],16.43B1A8	43B1A8:L"헛\n"
004015D2	C70424 C0334400	mov dword ptr ss:[esp],16.4433C0	4433C0:&"쑤B"
004015D9	E8 528D0200	call 16.42A330	
004015DE	C74424 04 D9004400	mov dword ptr ss:[esp+4],16.4400D9	4400D9:" Good Job!\n"

그다음에 password 입력후 사용자가 입력한 값 eax와 아까 계산된 ebp-3C를 비교해서 같지 않으면 밑의 wrong password로 내려가는 jne 40163C가 있고 같다면 Good Job을 보여주는것이라 생각하여

9-1. CodeEngn (basic L16)

```
ReWrit's Crackme#5
*****
* This is my 5th crackme, *
* i hope you will enjoy it. *
*****

Enter your Name: test
Enter your Password: test
```

직접 test라고 입력하여 ebp-38에 값을 넣고 계산된 ebp-3C값을 직접 보는것으로 했습니다.

```
명령: dword:[ebp-3C]
일지 중지됨 dword:[ebp-3C]=3D723D97 (1030897047)
```

Ebp-3C의 값 1030897047로 뜨는것을 확인했고 프로그램을 재시작하여 계산된값을 넣었습니다.

```
ReWrit's Crackme#5
*****
* This is my 5th crackme, *
* i hope you will enjoy it. *
*****

Enter your Name: test
Enter your Password: 103089704
7

Good Job!
=>|
```

옆 사진처럼 Good Job이라고 나온것을 확인하였습니다.



감사합니다

17기 최동현